

Modules

Most DCGM feature areas outside [Core Services](#) are implemented as shared- library modules. Core is module ID 0, is always loaded, and cannot be denylisted. A loadable module normally enters the process when an API or command first needs it; the NVSwitch module can also load automatically when supported hardware is present. Module availability still depends on hardware, plugins, external executables, privileges, and the host environment.

Use this table to choose the learning page for the workflow you need. For module names, states, denylist rules, lifetime, and exact syntax, see [dcgmi modules](#) and [nv-hostengine](#).

Loadable modules		
ID	Module	Functionality
1	NVSwitch and ConnectX	NVSwitch and ConnectX discovery, telemetry, health inputs, and link monitoring.
2	vGPU	Host-side NVIDIA vGPU inventory and telemetry.
3	Introspection	CPU and memory use of the host-engine process.
4	Health Monitoring	Passive watches that assess subsystem health from telemetry.
5	Policy	Notifications and actions in response to selected GPU conditions.
6	Configuration	Target GPU configuration and enforcement.
7	Diagnostics	Active, single-node deployment, hardware, integration, and stress diagnostics.
8	Profiling	Low-overhead profiling metrics and compatible metric groups.
9	CPU Monitoring	NVIDIA CPU and CPU-core discovery and telemetry.
10	Multi-Node Diagnostics	Coordinated diagnostics across supported multi-node systems.

Disabling Modules

Denylisting a module prevents it from loading and makes the corresponding functionality unavailable. It does not unload a module that is already resident. Use a runtime denylist when the host engine is already running and the target module has not been used; use a startup denylist when the module must be unavailable from the beginning of the process.

Runtime denylisting

List the modules before invoking the functionality that is to be disabled:

```
$ dcgmi modules --list
```

A `Not loaded` state is normal for a lazy-loaded module and means that the module is eligible for runtime denylisting. For example, denylist the Policy module before any client uses policy functionality, then list the states again to verify that it is `Denylisted`:

```
$ dcgmi modules --denylist Policy
$ dcgmi modules --list
```

The command accepts a case-insensitive module name or the numeric ID from the table above. It fails if the module is already loaded. A runtime denylist lasts for the current host-engine process and is not written to a persistent configuration file.

Startup denylisting

Pass numeric module IDs to `nv-hostengine` when the module must be disabled before clients can cause it to load. This example disables Introspection (3) and Health (4):

```
$ nv-hostengine --denylist-modules 3,4
```

Startup denylisting is especially important for the NVSwitch module because it can load automatically when supported hardware is present. Module ID 0 cannot be denylisted. To restore a denylisted module, restart the host engine without that module in the denylist.

The programmatic equivalent is `dcgmModuleDenylist()`. See [Modules](#) for the module API, [dcgmi modules](#) for runtime behavior, and [nv-hostengine](#) for startup behavior.



